

Project Documentation: FinTrack Engine

1. Project Overview & Objective

- **Project Name:** FinTrack: Real-Time Personal Finance & Stock Portfolio Engine
- **Programming Language:** C++
- **Objective:** The objective of this project is to provide users with an efficient platform to manage daily expenses, sort and search financial data seamlessly, and process real-time stock buy/sell orders in a systematic queue-based framework.

2. Problem Statement

Manual personal finance tracking often suffers from data and poor organization. Traditional approaches utilizing arrays are not very good. They require -defined memory allocation. This leads to RAM wastage and potential buffer overflows.

The FinTrack Engine is designed to overcome these limitations.

- The FinTrack Engine uses Dynamic Memory Allocation. This means the FinTrack Engine allocates memory at runtime. The memory allocation scales with the volume of data.
- The FinTrack Engine optimizes core operations. This means the FinTrack Engine can insert, sort data very quickly. The FinTrack Engine achieves minimal time complexity. The goal is to achieve $O(\log n)$ time complexity where applicable.

3. System Architecture & Data Structures Used

The FinTrack Engine uses four core data structures to achieve low-level efficiency and avoid overhead.

A. Singly Linked List (Transaction History)

- **Purpose:** To maintain a running ledger of all expense and income records.

- Logic: Every new transaction node is inserted at the **head** of the list. This ensures that the most recent transaction is always displayed first.
- Time Complexity: for Insertion: $O(1)$ and for Display: $O(n)$

B. Pointer-Based Stack (Undo System)

- Purpose: To track user operations and provide an "Undo" feature to reverse the last action.
- Logic: Operates on the LIFO (Last In, First Out) principle. The pushAction function logs an event to the stack, while popAction removes and discards the most recent event from memory.
- Time Complexity: for Push: $O(1)$ and for Pop: $O(1)$

C. Binary Search Tree - BST (Expense Sorting & Analysis)

- Purpose: To dynamically sort financial transactions based on monetary value for quick analytical queries.
- Logic: Financial records are sorted hierarchically; values smaller than the current node flow into the left subtree, while larger values flow into the right subtree. Sorting is achieved using an In-order Traversal (Left-Root-Right).
- Time Complexity: $O(\log n)$

D. Two-Pointer Queue (Stock Order Processing Engine)

- Purpose: To handle real-time stock buy and sell order requests within the portfolio module.
- Logic: Operates strictly on the FIFO (First In, First Out) principle. Incoming orders are enqueued at the rear pointer, and order executions are dequeued from the front pointer to preserve arrival order.
- Time Complexity: Enqueue: $O(1)$ and Dequeue: $O(1)$

5. Source Code & Implementation

<https://github.com/SafaSabir/FinTrack-DSA-Project>

6. Sample Test Cases (Inputs & Outputs)

- Scenario 1: Adding Expense

```
=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 1
Enter Description (e.g., Food, Salary): Grociers
Enter Amount: 1100
[SUCCESS] Transaction Added: Grociers (Rs. 1100)
```

- Scenario 2: View Transaction History

```
=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 2
--- Transaction History ---
- Grociers: Rs. 1100
- internet: Rs. 2500
- rent: Rs. 2000
- fuel: Rs. 1000
- food: Rs. 500
```

- Scenario 3: Undo Last Action

```

=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 3
[UNDO] Reversed action: Added Transaction (Groceries)

```

- **Scenario 4: View Sorted Tree**

```

=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 4

Sorted Expenses (Low to High): Rs. 500 | Rs. 1000 | Rs. 1100 | Rs. 2000 | Rs. 2500 |

```

- **Scenario 5: Place Stock Order**

```

=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 5
Enter Stock Ticker (e.g., AAPL, GOOG): AAPL
Enter Order Type (Buy/Sell): Buy
[QUEUE] Order placed in queue: Buy AAPL

```

- Scenario 6: Process Next Stock Order

```

=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 6

[PROCESSING] Executing Buy order for stock: AAPL...
[SUCCESS] Order Executed!

```

- Scenario 7: View Pending Stock Orders

```
=====
      FinTrack: Personal Finance
=====
1. Add Expense/Income (Linked List)
2. View All Transactions (Linked List Display)
3. Undo Last Action (Stack)
4. View Sorted Expenses (Tree Traversal)
5. Place Stock Order (Queue Enqueue)
6. Process Next Stock Order (Queue Dequeue)
7. View Pending Stock Orders (Queue Display)
0. Exit Application
-----
Enter your choice: 7

--- Pending Stock Orders ---
[BUY] -> TSLA
[Sell] -> GOOG
```

7. Conclusion

FinTrack shows how to use core computer science data structures in real financial software without using any heavy frameworks.